002 - Modelling and implementing a distributed system

1) Distributed Vote Count

This voting center node is responsible for simulating a vote. A vote is composed of several parties, the same for each center, and a favourability meter. This favourability meter is created in a subclass of the client class and is different for each client. The subclass of the client is also in charge of determining the vote of each voter.

The central tallying node is responsible for receiving the data sent by the clients and aggregating it to obtain the result. The tallying node uses regex and specific words to pick the right information for each center. After that an aggregator puts the result into different forms.

The voting center communicates with the server using text sent via the socket output. The advantage of using text to communicates is that the data sent will be treated only when the server can do it without modifying the order or the content. The server knows the end of a communication because of the word “END” written at the end of each communication. I also used regex to split the results so that the server can recreate the voting result of each center. To capture this communication, I used another thread to capture user input to start the aggregator and stop each center. To differentiate the different centers I pass the center ID in the arguments of the main class alongside the server IP.

To store the vote of each center I used a ConcurrentHashMap [1] this object is thread safe and ensure that only one thread can modify it at once.

The benefit of this design lies in its realism with the favourability meter, which integrate unpredicted result into the simulation, and the use of 10,000 votes each decided independently. But one of the flaws of this design is that the data sent by the client need to be strictly formatted so that the server can understand the right data.

CentralVotingServer VotingCenterClient

```
/opt/homebrew/opt/java/libexec/openjdk.jdk/Contents/Home/bin/java -javaagent:/Applicati
The voting aggregator server is running...
Type "end" after all the voting center sent their result (you need at least 1 result)

Receiving data from center Center1

A: 2463 votes
B: 493 votes
C: 1416 votes
D: 2090 votes
E: 3538 votes
Receiving data from center Center2

A: 1031 votes
B: 718 votes
C: 1942 votes
D: 507 votes
E: 5802 votes
Receiving data from center Center3

A: 1012 votes
B: 1966 votes
C: 1296 votes
D: 1346 votes
E: 4380 votes
Receiving data from center Center4

A: 1282 votes
B: 464 votes
C: 2398 votes
D: 1820 votes
E: 4036 votes
```

CentralVotingServer VotingCente

```
Receiving data from center Center3

A: 1012 votes
B: 1966 votes
C: 1296 votes
D: 1346 votes
E: 4380 votes
Receiving data from center Center4

A: 1282 votes
B: 464 votes
C: 2398 votes
D: 1820 votes
E: 4036 votes
end
Voting finished
Winner: E with 17756 votes
Result:
A: 14.47%
B: 9.1025%
C: 17.63%
D: 14.4075%
E: 44.39%

Result:
A: 5788
B: 3641
C: 7052
D: 5763
E: 17756
```

```
CentralVotingServer   VotingCenterClient

/opt/homebrew/opt/java/libexec/openjdk.jdk/Contents/Hom
Favorability meter: [13, 18, 42, 60, 100]
Center:Center4
A:1282 votes
B:464 votes
C:2398 votes
D:1820 votes
E:4036 votes
Results sent!

Process finished with exit code 0
```

Here is an example of the output for the voting client with the favourability meter, the vote of each party and the center ID.

2) Distributed numerical Integration

The integration server is responsible for calling a class to initialize the function, the range and the number of subintervals. This class verifies the input by avoiding inputs where a number isn't given or when the value of the upper range is less than the lower range. There is also a standard initialization available with "test" as arguments.

After the initialisation, I wait for 2 clients and assign them the function, the range, the number of subintervals and the name of the method that will be used. Each client is represented by a client manager; the manager can transform and send the function information into readable format for the client After starting 2 threads for the clients, I used a lock to start the threads at the same time. After starting the thread, I send a message to the client that then starts the calculation. When the calculation is finished, the results are sent to the client manager who stores it in a custom class.

The integration client is first calling an IntegrationDoer class, this class takes the data sent by the server and transform it so that those data can be used to compute the result. After transforming the data an IntegrationMethod object is created with the name

of the method. After creating the IntegrationMethod the calculations start at the same time as the time counter if the 2 threads are started. After the result is produced, it is sent to the server alongside the duration of execution and the name of the IntegrationMethod.

The integrationMethod class has an abstract function which is overridden by other classes implementing specific integration method and a method to evaluate the value of a function.

To start the threads at the same time I used a synchronized lock and “NotifyAll()” to broadcast the information from the main class of the server. And for the communication mechanisms I used the same as in the previous problem.

The benefit of this design is its maintainability with the strategy pattern for choosing the integration methods and the simplicity of adding new integration methods. But its limitation is the network communication because of the request sent to each client every 0.5 second.

```
IntegrationServer   IntegrationClient   IntegrationClient

/opt/homebrew/opt/java/libexec/openjdk.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA.app/
Listening on port 59898
THE FUNCTION:
Choose the value of each coefficient starting from the unit one and type "end" to finish your function:
1
x:1
x^2:end
f(x) = 1.0 +x

THE RANGE:
Choose the range of the function:
a=1
b=10
THE SUBDIVISION:
Choose the number of sub division that will be used in this integration:
100
Waiting for client
Client added succesfully
Starting the calculi
Starting the calculi

Ranked by time
Method: trapezoide
Value: 58.499999999999986
Duration: 76000 ns
```

```
IntegrationServer   IntegrationClient   IntegrationClient
/opt/homebrew/opt/java/libexec/openjdk.jdk/Contents/Home/bin/java
monteCarlo:
Result: 59.80397946258521
Time: 106000 ns
```

```
IntegrationServer   IntegrationClient   IntegrationClient
/opt/homebrew/opt/java/libexec/openjdk.jdk/Contents/Home/bin/java
trapezoide:
Result: 58.499999999999986
Time: 76000 ns
```

On the previous screenshot we can see the execution on the function f(x)=1+x with a small number of subintervals. We can see that the results are fairly different

IntegrationServer × IntegrationClient × IntegrationClient ×

```
/opt/homebrew/opt/java/libexec/openjdk.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA.
Listening on port 59898
THE FUNCTION:
Choose the value of each coefficient starting from the unit one and type "end" to finish your function:
1
x:1
x^2:end
f(x) = 1.0 +x

THE RANGE:
Choose the range of the function:
a=1
b=10
THE SUBDIVISION:
Choose the number of sub division that will be used in this integration:
1000000
Waiting for client
Client added succesfully
Starting the calculi
Starting the calculi

Ranked by time
Method: trapezoide
Value: 58.50000000000134
Duration: 5402000 ns
```

IntegrationServer × IntegrationClient × IntegrationClient ×

```
/opt/homebrew/opt/java/libexec/openjdk.jdk/Contents/Home/bin/jav
monteCarlo:
Result: 58.475646836884486
Time: 10771000 ns
```

IntegrationServer × IntegrationClient × IntegrationClient ×

```
/opt/homebrew/opt/java/libexec/openjdk.jdk/Contents/Home/bin/jav
trapezoide:
Result: 58.50000000000134
Time: 5402000 ns
```

On the previous screenshot we can see the same function but with a lot more of subIntervals so the 2 methods 's result are fairly identical.

```
Listening on port 59898
THE FUNCTION:
Choose the value of each coefficient starting from the unit one and type "end" to finish your function:
dkhvs
Invalid input. Please try again.
1
x:1
x^2:kdsjb
Invalid input. Please try again.
x^2:end
f(x) = 1.0 +x

THE RANGE:
Choose the range of the function:
a=fgd
Invalid input. Please try again.
a=1
b=-4
Invalid input. Please try again.
b=sdfj
Invalid input. Please try again.
b=10
THE SUBDIVISION:
Choose the number of sub division that will be used in this integration:
1000000
Waiting for client
Client added succesfully
Starting the calculi
Starting the calculi

Ranked by time
Method: trapezoide
Value: 58.50000000000134
Duration: 4743000 ns
```

```
IntegrationServer   IntegrationClient   IntegrationClient
/opt/homebrew/opt/java/libexec/openjdk.jdk/Contents/Home/bin/java
monteCarlo:
Result: 58.50070681215611
Time: 10579000 ns
```

```
IntegrationServer   IntegrationClient   IntegrationClient
/opt/homebrew/opt/java/libexec/openjdk.jdk/Contents/Home/bin/java
trapezoide:
Result: 58.50000000000134
Time: 4743000 ns
```

In the previous screenshot we can see how the errors are handled with the same situation as before.

```
Listening on port 59898
THE FUNCTION:
Choose the value of each coefficient starting from the unit one and type "end" to finish your function:
1
x:1
x^2:end
f(x) = 1.0 +x

THE RANGE:
Choose the range of the function:
a=1
b=5
THE SUBDIVISION:
Choose the number of sub division that will be used in this integration:
1000000
Waiting for client
Client added succesfully
Starting the calculi
Starting the calculi

Ranked by time
Method: trapezoide
Value: 16.00000000000001
Duration: 4819000 ns
```

IntegrationServer IntegrationClient IntegrationClient

```
/opt/homebrew/opt/java/libexec/openjdk.jdk/Contents/Home/bin/java
monteCarlo:
Result: 16.001038821413218
Time: 11190000 ns
```

IntegrationServer IntegrationClient IntegrationClient

```
/opt/homebrew/opt/java/libexec/openjdk.jdk/Contents/Home/bin/java
trapezoide:
Result: 16.00000000000001
Time: 4819000 ns
```

On the previous screenshot we can see the same function with a different range and the same number of subintervals.

IntegrationServer | IntegrationClient | IntegrationClient

```
/opt/homebrew/opt/java/libexec/openjdk.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA.a
Listening on port 59898
THE FUNCTION:
Choose the value of each coefficient starting from the unit one and type "end" to finish your function:
1
x:2
x^2:3
x^3:4
x^4:5
x^5:6
x^6:7
x^7:end
f(x) = 1.0 +2.0*x +3.0*x^2 +4.0*x^3 +5.0*x^4 +6.0*x^5 +7.0*x^6

THE RANGE:
Choose the range of the function:
a=1
b=10
THE SUBDIVISION:
Choose the number of sub division that will be used in this integration:
100000000

Waiting for client
Client added succesfully
Starting the calculi
Starting the calculi

Ranked by time
Method: trapezoide
Value: 1.1111102999999112E7
Duration: 353804000 ns
```

```
IntegrationServer   IntegrationClient   IntegrationClient
/opt/homebrew/opt/java/libexec/openjdk.jdk/Contents/Home/bin/java
monteCarlo:
Result: 1.111090995994257E7
Time: 809875000 ns
```

```
IntegrationServer   IntegrationClient   IntegrationClient
/opt/homebrew/opt/java/libexec/openjdk.jdk/Contents/Home/bin/java
trapezoide:
Result: 1.1111102999999112E7
Time: 353804000 ns
```

On the previous screenshot we can see another function being integrated.

3) Load Balancing

"Load balancing distributes network traffic" [2] between several servers to optimize resources and increase the quality of service. Load balancing can affect performance, scalability, availability and security.
Because the requests are distributed between several servers the chance of a server becoming saturated decreases and several tasks can be done in parallel. These two points speed up the response time and reduce bottlenecks of the service.

Load balancing affects scalability because if the service needs more hardware, add new servers and let the load balancer distribute the request between those servers. The same goes with when a server does not need as much power as it has, the service can reduce resources and save cost.

Load balancing also affects availability because if one server goes down for any reason the service will continue running with less performance. As for fault tolerance the load balancer can detect failures and redirect requests to healthy servers.

And finally, the load balancer can affect security because it can hide the service architecture from outsiders and analyse traffic to redirect it or modify it. These benefits help prevent cyberattacks like a Distributed Denial of Service (DDoS) which consists of overloading a service with thousands of connections at the same time.

There are several common implementations of load balancing like the “Round Robin”, the “Random with two choices”, the “Least connections”, the “Least Time”, the “URL Hash”, the “Source Ip Hash” and the “Consistent Hashing” Algorithm [2].

The Round Robin Algorithm works by sending “traffic to a list of servers in rotation can be implemented using Domain Name System (DNS)” [2]. This option is very easy to implement but it does not account for server capacity or server load.

The Random with two choices Algorithm works by selecting two servers and then using other algorithm like the least time or the least connections algorithm. This method can be useful with many servers but because of the random factor the two servers chosen can be overloaded [2].

The Least Connections Algorithm works by keeping track on the number of connections or requests of each server and then redirecting the traffic to the server with the lowest number. This method takes into account the current workloads, but it does not take account for the request or connection time or resource, and it needs to be a little bit more complex [2].

The Least Time Algorithm works by keeping tracks on the request or connection time of each server and then redirecting the traffic on the server with the lowest response time. This method takes in account the current workloads, but it needs to add a lot of complexity to the load balancer [2].

The URL hash Algorithm works by generating a hash value for each URL, then uses this generated value to choose the right server. This method is useful to avoid having user-specific data on every server. This can be helpful for ensuring that a client can access his shopping cart while not having his shopping cart on every server [2].
The Source Ip Hash works by using a hashing algorithm on the client’s IP adress to choose a server. The chosen server will be the only one used for this client. This method has the same benefits as the URL Hash Algorithm.

The Consistent Hashing Algorithm works by assigning a point on each server and user. All the point are then arranged in form of a ring. Each server has a limited capacity and handles the points representing user closest to it. When a user tries to connect and if the nearest server is at full capacity, the system tries to connect the user to the other server.

The Load Balancing Server first sends a signal to indicate that a server is trying to connect then it takes input sent by the load balancer. These inputs are either an end signal, an update signal, a message or a number. If the signal is an end signal a log entry is created and the server stops. If the signal is an update signal, the server sends

information about its run time like the number of tasks it has done. If the signal is a message either the message or a custom response is sent by the server. If the signal is a number, the server calculates the value of the Fibonacci sum for this number. Each signal is a string sent threw the socket and each server response is preceded by a custom code so the load balancer can treat them in the right way. In the server, to differentiate from a message and a number I try to transform the string into an integer, if an exception is raised, I know that it is a message and if not, I calculate the Fibonnaci value for this number.

The Load Balancing Client first sends a message to indicate that a client is trying to connect then it lets the user choose an ID; it shows the command. After that a thread that reads the message sent by the load Balancer and write the appropriate message is started. During this thread execution log entries are created and sent to the load balancer. After the thread creation the main loop of this class creates a requestId and taking the user input to send a request to the load balancer. If the user type nothing I send the text “EMPTY”. I create the requestId using an AtomicInteger [3] to create a counter that is thread safe and then to ensure the unicity of this Id I concatenate the clientId et the number given by the counter.

The LoadBalancer starts by instantiating the logger, this is where one can change the name of the file, a threadPool with a maximum of 25 thread is created. The first thread added in the thread pool is the Updater thread, it sends to all the server a request for the server information every 0.5 seconds, the next thread of the pool are listener threads, it is a thread for each client or server. In the listener thread information about the connecting socket is collected, for both of them the method to send them a message is stored, for a server the number of tasks is saved on a map. In this thread all the messages sent by the server and the client are treated. These messages are identified by an identifier like “UPD”, “RES”, “REQ”, “LOG”, “MES”.

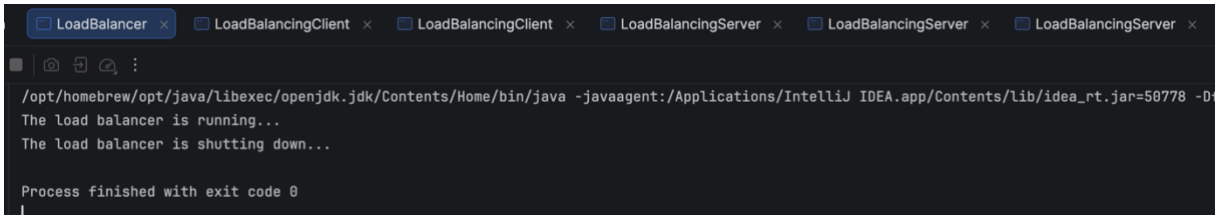
In the UPD identifier, the server sends the number of tasks they have done and then the load balancer saves it.

In the RES identifier, the server sends the results of a Fibonacci request, verify that this is a number by trying to parse it to Long. With the requestId in the message sent by the server the load balancer found the client who asked the result and then it sends a message to the client with the result.

In the REQ identifier, the client sends a message to the load balancer. The message can be “EMPTY”, “end” or any other valid request, if the message is “EMPTY” nothing is done. If the message is “end” the load balancer stops every other program, then it stops itself after writing the log file. If the message is neither of them, the server with the least number of tasks done is chosen to handle the current request.

In the LOG identifier, the load balancer wrote the chosen log using the logger class.

In the MES identifier, the server sends a message to the load balancer who choose the right client using the requestId and then sent the data of the message to the client.



LoadBalancer | LoadBalancingClient | LoadBalancingClient | LoadBalancingServer | LoadBalancingServer | LoadBalancingServer

```
/opt/homebrew/opt/java/libexec/openjdk.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA.app/Contents/lib/idea_rt.jar=50783 -D
Which Id do you want to have ?
client1
You can simulate user input with the console
Command:
- Type a number to calculate Fibonacci of this value
- Type a message to maybe obtain a secret response
- Type "end" to stop the load balancing
fdsbj
Message received: fdsbj
vcnb
Message received: vcnb
`
Message received: `

43
Result for Fibonnaci: 701408733
tggf
Message received: tggf
12
Result for Fibonnaci: 233
43
Result for Fibonnaci: 701408733
end
Simulation finishing

Process finished with exit code 0
```

LoadBalancer | LoadBalancingClient | LoadBalancingClient | LoadBalancingServer | LoadBalancingServer | LoadBalancingServer

```
client2
You can simulate user input with the console
Command:
- Type a number to calculate Fibonacci of this value
- Type a message to maybe obtain a secret response
- Type "end" to stop the load balancing
fdskgh
Message received: fdskgh
fshdhk
Message received: fshdhk
fkdghs
Message received: fkdghs
rhf
Message received: rhf
cgvx
Message received: cgvx
fbdh
Message received: fbdh
13
Result for Fibonnaci: 377
("
Message received: ("


5
Result for Fibonnaci: 8
45
Result for Fibonnaci: 1836311903
34
Result for Fibonnaci: 9227465
Simulation finishing

Process finished with exit code 0
```

LoadBalancer × LoadBalancingClient × LoadBalancingClient × LoadBalancingServer × LoadBalancingServer × LoadBalancingServer ×

```
/opt/homebrew/opt/java/libexec/openjdk.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA.app/Contents/lib/idea_rt.jar=50795 -D
Message received = fshdhk
Message sent: fshdhk


Message received = cgvx
Message sent: cgvx


Message received = ("
Message sent: ("


Starting the calculi of fibonacci for 34
Fibonacci(34) = 9227465


Message received = `
Message sent: `


Starting the calculi of fibonacci for 12
Fibonacci(12) = 233


Server stopped

Process finished with exit code 0
```

LoadBalancer × LoadBalancingClient × LoadBalancingClient × LoadBalancingServer × LoadBalancingServer × LoadBalancingServer ×

```
/opt/homebrew/opt/java/libexec/openjdk.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA.app/Contents/lib/idea_rt.jar=50801 -D
Message received = fkdghs
Message sent: fkdghs


Message received = fbdh
Message sent: fbdh


Starting the calculi of fibonacci for 5
Fibonacci(5) = 8


Message received = fdsbj
Message sent: fdsbj


Starting the calculi of fibonacci for 43
Fibonacci(43) = 701408733


Starting the calculi of fibonacci for 43
Fibonacci(43) = 701408733


Server stopped

Process finished with exit code 0
```

LoadBalancer × LoadBalancingClient × LoadBalancingClient × LoadBalancingServer × LoadBalancingServer × LoadBalancingServer ×

```
/opt/homebrew/opt/java/libexec/openjdk.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA.app/Contents/lib/idea_rt.jar=50807 -D
Message received = fdskgh
Message sent: fdskgh


Message received = rhf
Message sent: rhf


Starting the calculi of fibonacci for 13
Fibonacci(13) = 377


Starting the calculi of fibonacci for 45
Fibonacci(45) = 1836311903


Message received = vcnb
Message sent: vcnb


Message received = tggf
Message sent: tggf


Server stopped

Process finished with exit code 0
```

```
[16/04 16:21:59] client2[16/04 16:21:59] Socket[addr=/127.0.0.1,port=59898,localport=50808] -> Message received = fdskgh

[16/04 16:21:59] Socket[addr=/127.0.0.1,port=59898,localport=50808] -> Response = fdskgh
[16/04 16:21:59] load Balancer :client2-1 -> Message sent: fdskgh
[16/04 16:21:59] client2 -> Message received
[16/04 16:22:00] client2 -> Message sent
[16/04 16:22:00] Socket[addr=/127.0.0.1,port=59898,localport=50796] -> Message received = fshdhk
[16/04 16:22:00] Socket[addr=/127.0.0.1,port=59898,localport=50796] -> Response = fshdhk
[16/04 16:22:00] load Balancer :client2-2 -> Message sent: fshdhk
[16/04 16:22:00] client2 -> Message received
[16/04 16:22:01] client2 -> Message sent
[16/04 16:22:01] Socket[addr=/127.0.0.1,port=59898,localport=50803] -> Message received = fkdghs
[16/04 16:22:01] Socket[addr=/127.0.0.1,port=59898,localport=50803] -> Response = fkdghs
[16/04 16:22:01] load Balancer :client2-3 -> Message sent: fkdghs
[16/04 16:22:01] client2 -> Message received
[16/04 16:22:02] client2 -> Message sent
[16/04 16:22:02] Socket[addr=/127.0.0.1,port=59898,localport=50808] -> Message received = rhf
[16/04 16:22:02] Socket[addr=/127.0.0.1,port=59898,localport=50808] -> Response = rhf
[16/04 16:22:02] load Balancer :client2-4 -> Message sent: rhf
[16/04 16:22:02] client2 -> Message received
[16/04 16:22:03] client2 -> Message sent
[16/04 16:22:03] Socket[addr=/127.0.0.1,port=59898,localport=50796] -> Message received = cgvx
[16/04 16:22:03] Socket[addr=/127.0.0.1,port=59898,localport=50796] -> Response = cgvx
[16/04 16:22:03] load Balancer :client2-5 -> Message sent: cgvx
[16/04 16:22:03] client2 -> Message received
[16/04 16:22:05] client2 -> Message sent
[16/04 16:22:05] Socket[addr=/127.0.0.1,port=59898,localport=50803] -> Message received = fbdh
[16/04 16:22:05] Socket[addr=/127.0.0.1,port=59898,localport=50803] -> Response = fbdh
[16/04 16:22:05] load Balancer :client2-6 -> Message sent: fbdh
[16/04 16:22:05] client2 -> Message received
[16/04 16:22:06] client2 -> Message sent
[16/04 16:22:06] Socket[addr=/127.0.0.1,port=59898,localport=50808] -> Calculing fibonacci(13)
[16/04 16:22:06] Socket[addr=/127.0.0.1,port=59898,localport=50808] -> fibonacci(13) = 377
[16/04 16:22:06] load balancer: client2-7 -> Result Received: 377
[16/04 16:22:06] client2 -> Message received
[16/04 16:22:07] client2 -> Message sent
```

```
[16/04 16:22:06] load balancer: client2-7 -> Result Received: 377
[16/04 16:22:06] client2 -> Message received
[16/04 16:22:07] client2 -> Message sent
[16/04 16:22:07] Socket[addr=/127.0.0.1,port=59898,localport=50796] -> Message received = ("
[16/04 16:22:07] Socket[addr=/127.0.0.1,port=59898,localport=50796] -> Response = ("
[16/04 16:22:07] load Balancer :client2-8 -> Message sent: ("
[16/04 16:22:07] client2 -> Message received
[16/04 16:22:08] load balancer: client2-9 -> Empty request
[16/04 16:22:08] client2 -> Message sent
[16/04 16:22:08] load balancer: client2-10 -> Empty request
[16/04 16:22:08] client2 -> Message sent
[16/04 16:22:10] client2 -> Message sent
[16/04 16:22:10] Socket[addr=/127.0.0.1,port=59898,localport=50803] -> Calculing fibonacci(5)
[16/04 16:22:10] Socket[addr=/127.0.0.1,port=59898,localport=50803] -> fibonacci(5) = 8
[16/04 16:22:10] load balancer: client2-11 -> Result Received: 8
[16/04 16:22:10] client2 -> Message received
[16/04 16:22:11] client2 -> Message sent
[16/04 16:22:11] Socket[addr=/127.0.0.1,port=59898,localport=50808] -> Calculing fibonacci(45)
[16/04 16:22:11] Socket[addr=/127.0.0.1,port=59898,localport=50808] -> fibonacci(45) = 1836311903
[16/04 16:22:11] load balancer: client2-12 -> Result Received: 1836311903
[16/04 16:22:11] client2 -> Message received
[16/04 16:22:14] client2 -> Message sent
[16/04 16:22:14] Socket[addr=/127.0.0.1,port=59898,localport=50796] -> Calculing fibonacci(34)
[16/04 16:22:14] Socket[addr=/127.0.0.1,port=59898,localport=50796] -> fibonacci(34) = 9227465
[16/04 16:22:14] load balancer: client2-13 -> Result Received: 9227465
[16/04 16:22:14] client2 -> Message received
[16/04 16:22:16] client1 -> Message sent
[16/04 16:22:16] Socket[addr=/127.0.0.1,port=59898,localport=50803] -> Message received = fdsbj
[16/04 16:22:16] Socket[addr=/127.0.0.1,port=59898,localport=50803] -> Response = fdsbj
[16/04 16:22:16] load Balancer :client1-1 -> Message sent: fdsbj
[16/04 16:22:16] client1 -> Message received
[16/04 16:22:17] client1 -> Message sent
[16/04 16:22:17] Socket[addr=/127.0.0.1,port=59898,localport=50808] -> Message received = vcnb
[16/04 16:22:17] Socket[addr=/127.0.0.1,port=59898,localport=50808] -> Response = vcnb
[16/04 16:22:17] load Balancer :client1-2 -> Message sent: vcnb
[16/04 16:22:17] client1 -> Message received
```

```
[16/04 16:22:17] load Balancer :client1-2 -> Message sent: vcnb
[16/04 16:22:17] client1 -> Message received
[16/04 16:22:17] client1 -> Message sent
[16/04 16:22:17] Socket[addr=/127.0.0.1,port=59898,localport=50796] -> Message received = `
[16/04 16:22:17] Socket[addr=/127.0.0.1,port=59898,localport=50796] -> Response = `
[16/04 16:22:17] load Balancer :client1-3 -> Message sent: `
[16/04 16:22:17] client1 -> Message received
[16/04 16:22:17] load balancer: client1-4 -> Empty request
[16/04 16:22:17] client1 -> Message sent
[16/04 16:22:19] client1 -> Message sent
[16/04 16:22:19] Socket[addr=/127.0.0.1,port=59898,localport=50803] -> Calculing fibonacci(43)
[16/04 16:22:19] Socket[addr=/127.0.0.1,port=59898,localport=50803] -> fibonacci(43) = 701408733
[16/04 16:22:19] load balancer: client1-5 -> Result Received: 701408733
[16/04 16:22:19] client1 -> Message received
[16/04 16:22:20] client1 -> Message sent
[16/04 16:22:20] Socket[addr=/127.0.0.1,port=59898,localport=50808] -> Message received = tggf
[16/04 16:22:20] Socket[addr=/127.0.0.1,port=59898,localport=50808] -> Response = tggf
[16/04 16:22:20] load Balancer :client1-6 -> Message sent: tggf
[16/04 16:22:20] client1 -> Message received
[16/04 16:22:22] client1 -> Message sent
[16/04 16:22:22] Socket[addr=/127.0.0.1,port=59898,localport=50796] -> Calculing fibonacci(12)
[16/04 16:22:22] Socket[addr=/127.0.0.1,port=59898,localport=50796] -> fibonacci(12) = 233
[16/04 16:22:22] load balancer: client1-7 -> Result Received: 233
[16/04 16:22:22] client1 -> Message received
[16/04 16:22:23] client1 -> Message sent
[16/04 16:22:23] Socket[addr=/127.0.0.1,port=59898,localport=50803] -> Calculing fibonacci(43)
[16/04 16:22:23] Socket[addr=/127.0.0.1,port=59898,localport=50803] -> fibonacci(43) = 701408733
[16/04 16:22:23] load balancer: client1-8 -> Result Received: 701408733
[16/04 16:22:23] client1 -> Message received
[16/04 16:22:25] load balancer: client1-9 -> Simulation ending
[16/04 16:22:25] Socket[addr=/127.0.0.1,port=59898,localport=50808] -> Server closed
[16/04 16:22:25] Socket[addr=/127.0.0.1,port=59898,localport=50796] -> Server closed
[16/04 16:22:25] client2 -> Server closing
```

In the last 3 screen there is the log file of this simulation.

[1]"ConcurrentHashMap (Java Platform SE 8)." Accessed: Apr. 14, 2026. [Online]. Available: https://docs.oracle.com/javase/8/docs/api/java/util/concurrent/ConcurrentHashMap.html
[2]"Load Balancer," F5, Inc. Accessed: Apr. 16, 2026. [Online]. Available: https://www.f5.com/glossary/load-balancer
[3]"AtomicInteger (Java Platform SE 8)." Accessed: Apr. 16, 2026. [Online]. Available: https://docs.oracle.com/javase/8/docs/api/java/util/concurrent/atomic/AtomicInteger.html